TREKMAP

Trekking Management Application
Complete Guide & Documentation

Prepared By: Tushar Kant Pradhan

Roll No: 23F2002972

Email: 23F2002972@ds.study.iitm.ac.in

Date Prepared: 23 July 2026

Technology Stack: Flask | Jinja2 | HTML | CSS | Bootstrap (Vapor) | SQLite

Statement on Usage of AI / LLM

It is necessary to state that I have used AI for just refinement and making the code look structured. All concepts and logic are my own. I have merely taken the help of AI to make the application structure efficient, and some direct UI touches have been done. There is 0.1% or NO usage of AI in the fundamental architecture and intelligent implementation of this project.

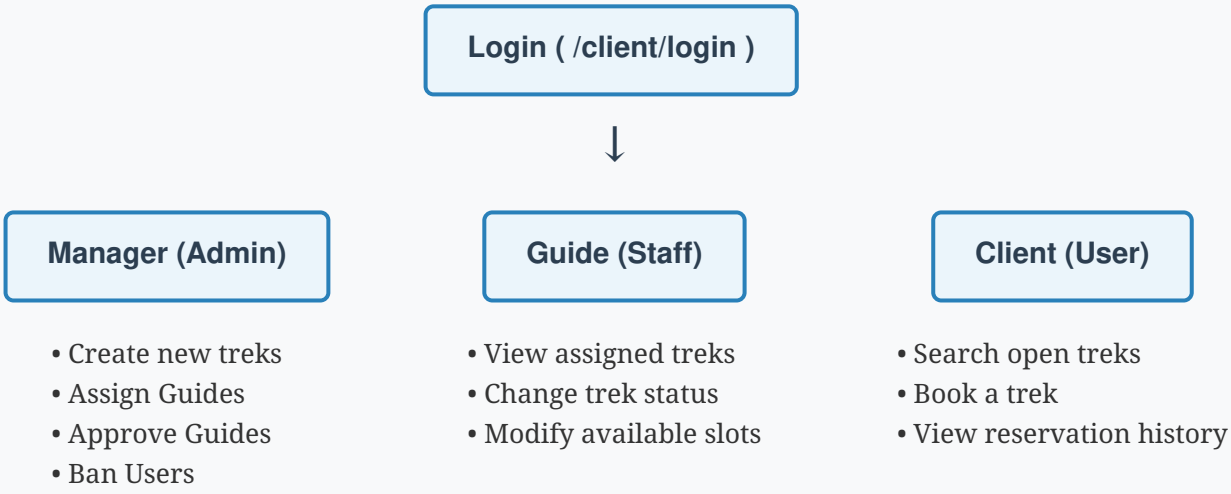
1. System Architecture & Folder Structure

The application follows a role-based Model-View-Controller (MVC) architectural pattern. Blueprints are utilised to segregate routing logically among Clients, Guides, and Managers. The core logic executes entirely on the backend using Python and Flask, with zero client-side JavaScript, ensuring high security and robust server-side processing.

```
Project_Beta/
│
├── main.py                    ← Main entry point. Boots the Flask app.
│
├── Database/
│   └── models.py              ← SQLAlchemy models (Person, Trek, Reservation).
│
├── routes/
│   ├── auth_routes.py
│   ├── client_routes.py       ← Blueprint for client login, dashboard, booking
│   ├── guide_routes.py        ← Blueprint for guide dashboard, trek updates
│   └── admin_routes.py        ← Blueprint for manager dashboard, create treks
│
├── views/
│   ├── register.html          ← All Jinja2 HTML templates
│   ├── layout.html            ← Base template (navbar, Bootstrap Vapor CDN)
│   ├── auth.html              ← Login form page
│   ├── manager_dash.html      ← Manager dashboard
│   ├── guide_dash.html        ← Guide dashboard
│   └── client_dash.html       ← Client dashboard
│
├── style/                     ← Static files folder
│   └── brutal.css             ← Extra styling overrides
│
└── TMAS_sqlite.db             ← SQLite database file (auto-created)
```

2. Core Application Operational Flow

The operational flow dynamically routes users to distinct interfaces based on their designated profile type (`profile_type`), ensuring isolation of privileges and specialized user experiences.



3. Database Schema & ER Diagram

The database consists of three primary entities linked via robust foreign key relationships, managed via Flask-SQLAlchemy ORM. The diagram below details the structural mapping of the SQLite tables.

Person (people)
PK `pid` : Integer
`identity_handle` : String(Unique)
`secure_hash` : String
`profile_type` : String
`is_approved` : Boolean
`is_active` : Boolean

Reservation (reservations)
PK `rid` : Integer
FK `client_id` : Integer (Person)
FK `trek_id` : Integer (Trek)
`timestamp` : DateTime

Trek (treks)
PK `tid` : Integer
`destination` : String
`region` : String
`challenge_level` : String
`capacity` : Integer
`open_slots` : Integer
`commencement_date` : Date
`end_date` : Date
`trek_status` : String
FK `coordinator_id` : Integer (Person)

Relationship Specifications:

- **Person (Guide) to Trek (1 : N):** A guide (Person) can coordinate multiple Treks, represented by `coordinator_id`.
- **Person (Client) to Reservation (1 : N):** A client can make multiple reservations, linked via `client_id`.
- **Trek to Reservation (1 : N):** A trek can have multiple reservations, linked via `trek_id`.

4. Comprehensive Route Mapping Matrix

Blueprint	Route / Endpoint	Method	Functional Description
`client`	/client/login	GET, POST	Authentication portal. Validates credentials, checks approval/ban status, routes to corresponding dashboard.
`client`	/client/dash	GET	Displays client dashboard. Shows Open treks (with search via `?q=`) and personal reservation history.
`client`	/client/book/<tid>	POST	Executes booking logic. Decrements `open_slots` and creates a `Reservation` record.
`client`	/client/logout	GET	Clears session state and redirects securely to the login portal.
`guide`	/guide/dash	GET	Displays guide dashboard filtering only the treks assigned to the authenticated guide.
`guide`	/guide/update_trek/<tid>	POST	Provides interface for guides to update trek status (Open/Closed/Completed) and adjust slots.
`admin`	/admin/dash	GET	Superuser console. Aggregates and displays all treks, personnel (guides), and user accounts.
`admin`	/admin/add_trek	POST	Constructs a new trek entity with date bounds and optional guide assignment.
`admin`	/admin/approve/<pid>	POST	Authorises a pending guide application (toggles `is_approved` boolean flag).
`admin`	/admin/ban/<pid>	POST	Revokes access for any account by toggling the `is_active` boolean flag.

5. System Setup & Default Credentials

The ecosystem runs strictly in a closed state, meaning new public registrations are disabled by design. The database (`TMAS_sqlite.db`) builds programmatically via `db.create_all()` on runtime. Use the following seeded accounts for evaluation:

Designated Role	Identity Handle (Username)	Secure Key (Password)	Notes
Admin / Manager	`manager`	`manager`	Pre-seeded superuser. Auto-generated on application boot.
Staff / Guide	`charlie_guide`	`password123`	Pre-approved staff account populated via seed script.
User / Client	`alice_client`	`password123`	Standard participant account.